Escolha de Case: Lead Scorer vs. Redesign de Suporte

https://gemini.google.com/app/50509ca0e32de3e7

*User prompt: Tenho um desafio em case para fazer. Preciso decidir entre 2 cases. Consegue me ajudar com esse ponto? Segue os dois cases, para analisar. Preciso que verifique os pontos positivos, a faciliade e o que encontraria mais relevancia. # Challenge 003 — Lead Scorer **Área:** Vendas / RevOps **Tipo:** Build (construir solução funcional) **Time budget:** 4-6 horas --- ## Contexto Você é o novo AI Master da área de **Vendas**. O time comercial tem 35 vendedores distribuídos em escritórios regionais, gerenciados por managers, trabalhando um pipeline de ~8.800 oportunidades. Hoje, a priorização é feita "no feeling" — cada vendedor decide quais deals focar com base na própria experiência e intuição. A Head de Revenue Operations te chamou e disse: > *"Nossos vendedores gastam tempo demais em deals que não vão fechar e deixam oportunidades boas esfriar. Preciso de algo funcional — não um modelo no Jupyter Notebook que ninguém vai usar. Quero uma ferramenta que o vendedor abra, veja o pipeline, e saiba onde focar. Pode ser simples, mas precisa funcionar."* Este é o challenge mais "mão na massa". O deliverable principal é **software funcionando** — não um documento. --- ## Dados disponíveis Quatro tabelas de um CRM, todas interconectadas: **Dataset:** [CRM Sales Predictive Analytics] (https://www.kaggle.com/datasets/agungpambudi/crm-sales-predictive-analytics) (licença CC0) | Arquivo | O que contém | Registros | Campos-chave | |---------|-------------|-----------|-------------| | `accounts.csv` | Contas clientes — setor, receita, número de funcionários, localização, empresa-mãe | ~85 | `account` | | `products.csv` | Catálogo de produtos com série e preço | 7 | `product` | | `sales_teams.csv` | Vendedores com seu manager e escritório regional | 35 | `sales_agent` | | `sales_pipeline.csv` | Pipeline completo — cada oportunidade com stage, datas, vendedor, produto, conta e valor de fechamento | ~8.800 | `opportunity_id` → liga tudo | ### Estrutura dos dados ``` accounts ←── sales_pipeline ──→ products ↓ sales_teams ``` O `sales_pipeline.csv` é a tabela central. Cada registro é uma oportunidade com: - `deal_stage`: Prospecting, Engaging, Won, Lost - `engage_date` / `close_date`: timeline do deal - `close_value`: valor real de fechamento (0 se Lost) --- ## O que entregar ### 1. Solução funcional (obrigatório) Construa algo que um vendedor possa usar. Não importa a tecnologia — importa que funcione. **Exemplos de soluções válidas:** - Aplicação web (Streamlit, React, HTML+JS, qualquer coisa) - Dashboard interativo (Plotly Dash, Retool, Metabase) - CLI tool ou script que gera relatório priorizados - API que recebe dados de um deal e retorna score + explicação - Planilha inteligente com fórmulas de scoring - Bot que envia prioridades por Slack/email **Requisitos mínimos:** - Precisa **rodar** (não é mockup, wireframe ou PowerPoint) - Precisa usar os **dados reais** do dataset - Precisa ter **lógica de scoring/priorização** (não é só ordenar por valor) - O vendedor precisa entender **por que** um deal tem score alto ou baixo ### 2. Documentação mínima (obrigatório) - **Setup:** Como rodar a solução (dependências, comandos, URL) - **Lógica:** Que critérios de scoring você usou e por quê - **Limitações:** O que a solução não faz e o que precisaria pra escalar ### 3. Process log (obrigatório) Evidências de como você usou IA para construir. Leia o [Guia de Submissão](../../submission-guide.md). Este challenge é especialmente interessante para quem usa "vibe coding" — Cursor, Claude Code, Replit Agent, v0, etc. **Mostre o processo.** --- ## Critérios de qualidade - A solução **funciona de verdade**? Dá pra rodar seguindo as instruções? - O scoring faz sentido? Usa as features certas? Vai além do óbvio? - O vendedor (não-técnico) consegue usar e entender? - A interface ajuda a tomar decisão ou só mostra dados? - O código é limpo o suficiente pra outro dev dar manutenção? --- ## Dicas - A Head de RevOps não pediu ML perfeito. Pediu algo **útil**. Comece simples, itere. - Deal stage, tempo no pipeline, tamanho da conta, produto e vendedor são features óbvias. O que mais importa? Olhe os dados. - Um scoring baseado em regras + heurísticas, bem apresentado, vale mais que um XGBoost sem interface. - **Explainability ganha.** Se o vendedor entender POR QUE o deal tem score 85, a ferramenta é 10x mais útil que um número sem contexto. - Pense no uso real: o vendedor abre isso na segunda-feira de manhã. O que ele precisa ver? - Bonus: se a*

*solução tiver filtro por vendedor/manager/região, fica imediatamente mais útil. # Challenge 002 — Redesign de Suporte **Área:** Operações / CX **Tipo:** Diagnóstico + Automação + Build **Time budget:** 4-6 horas --- ## Contexto Você é o novo AI Master da área de **Suporte ao Cliente** de uma empresa de tecnologia. A operação atende ~30.000 tickets por ano via email, chat, telefone e redes sociais. O time está sobrecarregado, o tempo de resolução subiu, e a satisfação do cliente caiu. O Diretor de Operações te chamou e disse: > *"Quero que você olhe nossos dados de suporte e me diga três coisas: onde estamos perdendo tempo, o que pode ser automatizado com IA, e me mostre que funciona — não quero só um PowerPoint. Quero ver algo rodando."* Este challenge testa três habilidades ao mesmo tempo: capacidade analítica (diagnóstico), pensamento de processo (o que automatizar e o que não), e capacidade de construir (protótipo funcional). --- ## Dados disponíveis Dois datasets complementares, ambos públicos no Kaggle: ### Dataset 1 — Métricas operacionais + texto de tickets **Dataset:** [Customer Support Ticket Dataset](https://www.kaggle.com/datasets/suraj520/customer-support-ticket-dataset) (licença CC0) | Coluna | Descrição | |--------|-----------| | `Ticket ID` | Identificador único | | `Customer Name`, `Email`, `Age`, `Gender` | Dados do cliente | | `Product Purchased` | Produto relacionado ao chamado | | `Ticket Type` | Tipo (Technical issue, Billing inquiry, Product inquiry) | | `Ticket Subject` | Assunto resumido | | `Ticket Description` | **Texto completo** da reclamação/pedido do cliente | | `Ticket Status` | Open, Closed, Pending customer response | | `Resolution` | **Texto da resolução** aplicada pelo agente | | `Ticket Priority` | Low, Medium, High, Critical | | `Ticket Channel` | Email, Phone, Chat, Social media | | `First Response Time` | Tempo até primeira resposta | | `Time to Resolution` | Tempo total até resolução | | `Customer Satisfaction Rating` | Nota de satisfação do cliente | **~30.000 registros** com texto real de descrição e resolução. ### Dataset 2 — Classificação de tickets IT **Dataset:** [IT Service Ticket Classification Dataset] (https://www.kaggle.com/datasets/adisongoh/it-service-ticket-classification-dataset) (licença CC0) | Coluna | Descrição | |--------|-----------| | `Document` | **Texto completo** do ticket de suporte | | `Topic_group` | Classificação em 8 categorias (Hardware, HR Support, Access, Storage, Purchase, etc.) | **~48.000 registros** com texto real de tickets de uma organização. --- ## O que entregar ### 1. Diagnóstico operacional (obrigatório) Com base no Dataset 1, identifique com dados: - **Onde o fluxo trava?** Gargalos por canal, prioridade, tipo de ticket. Quais combinações geram os piores tempos de resolução? - **O que impacta satisfação?** Quais variáveis mais influenciam o `Customer Satisfaction Rating`? É o tempo de resposta? O canal? O tipo de problema? - **Quanto estamos desperdiçando?** Quantifique em horas e, se possível, estime custo. Onde está o maior desperdício recuperável? ### 2. Proposta de automação com IA (obrigatório) Com base em ambos os datasets, proponha: - **O que automatizar** — Classificação automática, roteamento inteligente, respostas sugeridas, triagem de prioridade, detecção de duplicatas — o que fizer sentido com base nos dados. - **O que NÃO automatizar** — Nem tudo deve ser delegado para IA. Quais tipos de ticket exigem julgamento humano? Justifique com exemplos dos dados. - **Como funcionaria na prática** — Descreva o fluxo proposto: ticket entra → o que acontece em cada etapa → onde IA atua → onde humano intervém. ### 3. Protótipo funcional (diferencial) Construa algo que demonstre que sua proposta funciona. Exemplos: - Um classificador de tickets que recebe texto e retorna categoria + confiança - Um sistema de respostas sugeridas baseado em tickets similares - Um roteador automático que categoriza e prioriza tickets novos - Um dashboard que mostra os gargalos em tempo real - Qualquer coisa funcional que demonstre a automação proposta ### 4. Process log (obrigatório) Evidências de como você usou IA. Leia o [Guia de Submissão](../../submission-guide.md). --- ## Critérios de qualidade - Usou ambos os datasets? (um tem métricas, outro tem texto — o poder está no cruzamento) - O diagnóstico tem números concretos ou é genérico? - A proposta de automação é realista? (automatizar 100% é red flag, não virtude) - Sabe distinguir onde IA ajuda de onde humano é insubstituível? - Se construiu protótipo: funciona com dados reais, não com 3 exemplos cherry-picked? --- ## Dicas - O Dataset 1 tem métricas operacionais. O Dataset 2 tem 48K textos classificados em 8 categorias — excelente pra treinar/testar classificadores. - "Usar NLP para classificar tickets" é genérico. "Classificar tickets em 8 categorias com 92% de acurácia usando embeddings + zero-shot classification" é específico. - Pense no agente de suporte: o que facilitaria a vida dele no dia a dia? - O Diretor de Operações quer ver ROI. Se puder estimar "automação X economiza Y*

horas/mês", isso muda a conversa. - Cuidado com a armadilha de automatizar tudo — os melhores candidatos sabem onde parar.

Response: Ambos os cases são excelentes, mas a escolha ideal depende da **estratégia de entrega** que você quer adotar e de **onde você quer focar seu tempo de desenvolvimento** nas 4-6 horas disponíveis.

Abaixo está a análise comparativa entre o **Lead Scorer** e o **Redesign de Suporte**:

1. Challenge 003 — Lead Scorer (Vendas / RevOps)

Pontos Positivos

- **Foco Claro e Direto:** É um problema com escopo fechado e objetivo definido — entregar **software funcional com interface** para a equipe de vendas.
- **Excelente para *Vibe Coding*:** Com ferramentas como Cursor, v0 e Streamlit/React, é rápido prototipar um painel moderno com filtros por vendedor/região, cards de resumo e visualização de scoring.
- **Narrativa de Negócio Simples:** É fácil defender a lógica do scoring com regras de negócio + heurísticas simples (ex.: combinar *close_value*, setor da conta, tempo no *stage* e histórico do vendedor) acopladas a uma explicação visual (*explainability*).

⚡ Facilidade de Execução

- **Muito Alta (Nota: 9/10):** A estrutura relacional dos CSVs é simples (pipeline no centro ligando *accounts*, *products* e *teams*). Não exige processamento complexo de texto nem modelagem pesada de Machine Learning.

🎯 Relevância

- Demostra capacidade de **construir produtos orientados a dados (Data Products)** rápidos e úteis para o time comercial. O resultado final é visual e fácil de impressionar em uma apresentação.

2. Challenge 002 — Redesign de Suporte (CX / Operações)

Pontos Positivos

- **Escopo Abrangente:** Demonstra visão de negócios de ponta a ponta: diagnóstico com dados, cálculo de ROI/horas economizadas e arquitetura de processos (humano + IA).
- **Trabalho com Dados Não Estruturados (Texto/NLP):** Permite usar embeddings, *zero-shot classification* ou LLMs para categorização e geração de respostas.

⚡ Facilidade de Execução

- **Média / Baixa (Nota: 6/10):** O escopo é significativamente maior para o tempo estipulado. Exige cruzar **dois datasets distintos**, fazer diagnóstico quantitativo, desenhar a arquitetura do processo e ainda construir um protótipo funcional.

🎯 Relevância

- Demonstra visão estratégica avançada de **AI Operations** e inteligência analítica, mostrando que você sabe medir impacto financeiro/operacional e impor limites éticos/práticos à automação.

Comparativo Estruturado

Critério	Lead Scorer (RevOps)	Redesign de Suporte (CX)
Complexidade dos Dados	Baixa (4 CSVs relacionais simples)	Alta (2 datasets com textos longos e métricas)
Carga de Desenvolvimento	Concentrada na Interface + Lógica de Score	Dividida entre Análise/Diagnóstico + Protótipo
Risco de Atraso no Prazo	Baixo	Médio/Alto (escopo mais longo)
Maior Forte do Deliverable	Interface funcional, usabilidade e *vibe coding*	Diagnóstico de negócio, ROI e NLP/Arquitetura

Recomendação Final

- **Escolha o Lead Scorer se:** Seu foco for entregar um **aplicativo web/dashboard completo e fluido**, mostrando como acelerar a criação de interfaces úteis e bem explicadas usando *vibe coding* dentro da janela de 4-6 horas.
- **Escolha o Redesign de Suporte se:** Você quiser se posicionar de forma mais **estratégica e analítica**, mostrando domínio em cálculo de ROI, diagnósticos operacionais com dados e arquitetura de automação com LLMs/NLP.

  Veredito: Para garantir um entregável com **menor risco de estouro de escopo** e **máximo impacto visual/funcional no tempo limite**, o **Lead Scorer** costuma ser a opção mais segura e eficiente.
